

Lab Assignment 15.5

Name : Rishitha Kunchala

Hall Ticket no :2303A51333

Batch No : 20

Task -1:

Prompt: The task is to develop a Student Attendance RESTful API using Node.js and Express. The API should support endpoints for viewing attendance records, marking attendance, updating attendance records, and deleting attendance entries. The API must handle JSON-based requests and responses. The system should store attendance data in memory and allow CRUD operations through HTTP methods like GET, POST, PUT, and DELETE.

```
2 // import
3 const express = require('express');
4 const app = express();
5
6 // Middleware to parse JSON
7 app.use(express.json());
8
9 // In-memory data store
10 let attendanceRecords = [];
11 let nextId = 1;
12
13 // Root route
14 app.get('/', (req, res) => {
15   res.status(200).send('Attendance API is running');
16 });
17
18 // GET all attendance records
19 app.get('/attendance', (req, res) => {
20   res.status(200).json(attendanceRecords);
21 });
22
23 // POST add attendance record
24 app.post('/attendance', (req, res) => {
25   const { studentId, date, status } = req.body;
26
27   if (!studentId || !date || !status) {
28     return res.status(400).json({
29       message: 'Missing required fields: studentId, date, and status are required'
30     });
31   }
32
33   const normalizedStatus = status.toString().trim().toLowerCase();
34
35   if (normalizedStatus !== 'present' && normalizedStatus !== 'absent') {
36     return res.status(400).json({
37       message: "Status must be either 'present' or 'absent'"
38     });
39   }
40
41   const newRecord = {
42     id: nextId++,
43     studentId,
44     date,
```

OUTPUT:

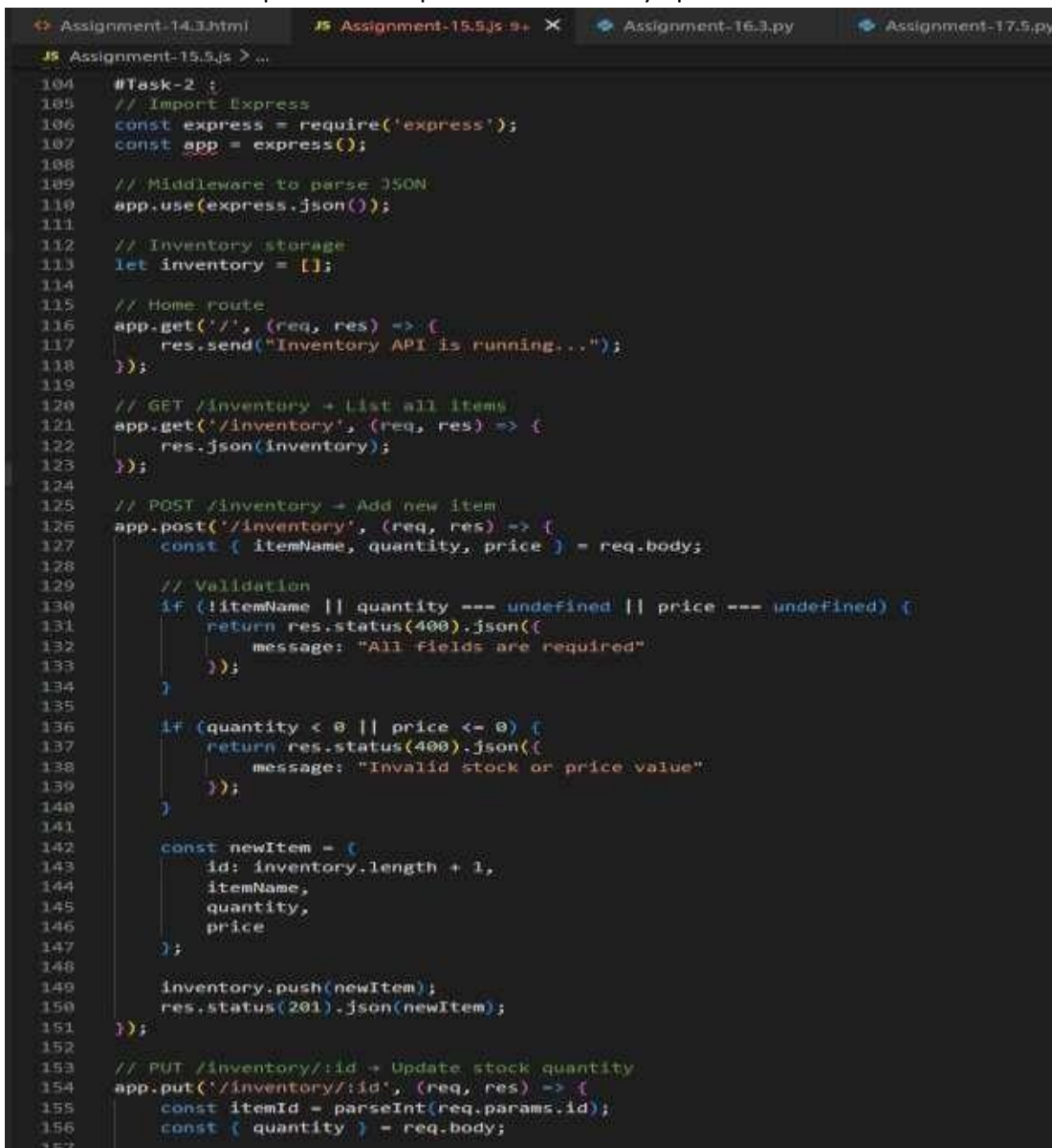
```
pretty:print
[{"id":1,"studentId":"301","date":"2026-04-01","status":"present"}, {"id":2,"studentId":"302","date":"2026-04-01","status":"absent"}, {"id":3,"studentId":"303","date":"2026-04-01","status":"absent"}]
```

Justification:

This project demonstrates how RESTful APIs can be built using Node.js and Express for managing student attendance records. It helps in understanding CRUD operations, JSON data handling, and API endpoint development. The system allows easy maintenance and updating of attendance records through HTTP requests. This project is useful for learning backend development and API design concepts.

Task 2:

Prompt: The task is to develop a REST API for warehouse inventory management using Node.js and Express. The API should provide endpoints to list all inventory items, add new items, update stock quantity, and delete items. The system should validate stock quantity and price values before updating the inventory. The API must handle JSON-based requests and responses for inventory operations.



```
Assignment-14.3.html JS Assignment-15.5.js a+ X Assignment-16.3.py Assignment-17.5.py
JS Assignment-15.5.js > ...
104 #Task-2 ;
105 // Import Express
106 const express = require('express');
107 const app = express();
108
109 // Middleware to parse JSON
110 app.use(express.json());
111
112 // Inventory storage
113 let inventory = [];
114
115 // Home route
116 app.get('/', (req, res) => {
117   res.send("Inventory API is running...");
118 });
119
120 // GET /inventory + List all items
121 app.get('/inventory', (req, res) => {
122   res.json(inventory);
123 });
124
125 // POST /inventory + Add new item
126 app.post('/inventory', (req, res) => {
127   const { itemName, quantity, price } = req.body;
128
129   // Validation
130   if (!itemName || quantity === undefined || price === undefined) {
131     return res.status(400).json({
132       message: "All fields are required"
133     });
134   }
135
136   if (quantity < 0 || price <= 0) {
137     return res.status(400).json({
138       message: "Invalid stock or price value"
139     });
140   }
141
142   const newItem = {
143     id: inventory.length + 1,
144     itemName,
145     quantity,
146     price
147   };
148
149   inventory.push(newItem);
150   res.status(201).json(newItem);
151 });
152
153 // PUT /inventory/:id + Update stock quantity
154 app.put('/inventory/:id', (req, res) => {
155   const itemId = parseInt(req.params.id);
156   const { quantity } = req.body;
157
```

Output:

Justification:

```
Pretty-print
[{"id":1,"itemName":"Laptop","quantity":10,"price":50000}, {"id":2,"itemName":"mouse","quantity":10,"price":5000}, {"id":3,"itemName":"cpu","quantity":10,"price":30000}]
```

This project demonstrates how REST APIs can be used to manage warehouse inventory and stock levels efficiently. It helps in understanding CRUD operations, data validation, and API development using Node.js and Express. The system ensures that invalid stock values are not entered through validation. This project is useful for learning backend development and inventory management systems.

Task 3:

Prompt: The task is to develop a RESTful API for hotel room booking using Node.js and Express. The API should allow users to view available rooms, book a room, view booking details by room ID, and cancel bookings. The system should handle errors such as booking unavailable rooms or invalid room IDs. The API must use JSON-based request and response handling.

```
JS Assignment-15.5.js >...
197 # Task-3 :
198 // Import Express
199 const express = require('express');
200 const app = express();
201
202 // Middleware to parse JSON
203 app.use(express.json());
204
205 // Sample rooms data
206 let rooms = [
207   { id: 1, roomNumber: "101", type: "Single", price: 1000, isBooked: false, customer: null },
208   { id: 2, roomNumber: "102", type: "Double", price: 2000, isBooked: false, customer: null },
209   { id: 3, roomNumber: "103", type: "Suite", price: 4000, isBooked: false, customer: null },
210   { id: 4, roomNumber: "104", type: "Single", price: 1200, isBooked: false, customer: null }
211 ];
212
213 // Home route
214 app.get('/', (req, res) => {
215   res.send("Hotel Booking API is running...");
216 });
217
218 // GET /rooms - View available rooms
219 app.get('/rooms', (req, res) => {
220   const availableRooms = rooms.filter(room => !room.isBooked);
221   res.json(availableRooms);
222 });
223
224 // POST /rooms/book - Book a room
225 app.post('/rooms/book', (req, res) => {
226   const { roomId, customerName } = req.body;
227
228   if (!roomId || !customerName) {
229     return res.status(400).json({
230       message: "roomId and customerName are required"
231     });
232   }
233
234   const room = rooms.find(r => r.id === roomId);
235
236   if (!room) {
237     return res.status(404).json({ message: "Room not found" });
238   }
239
240   if (room.isBooked) {
241     return res.status(400).json({ message: "Room already booked" });
242   }
243
244   room.isBooked = true;
245   room.customer = customerName;
246
247   res.status(201).json({
248     message: "Room booked successfully",
249     room
250   });
251 }
```

Output:

Justification:

```

Pretty-print
[{"id":1,"roomNumber":101,"type":"Single","price":1000,"isBooked":false,"customer":null}, {"id":2,"roomNumber":102,"type":"Double","price":2000,"isBooked":false,"customer":null}, {"id":3,"roomNumber":103,"type":"Suite","price":4000,"isBooked":false,"customer":null}, {"id":4,"roomNumber":104,"type":"Single","price":1200,"isBooked":false,"customer":null}]

```

This project demonstrates how RESTful APIs can be used to simulate hotel room booking operations. It helps in understanding API routing, booking logic, and error handling in backend development. The system ensures that rooms cannot be double booked and handles invalid booking requests. This project is useful for learning real-world booking system API development using Node.js and Express.

Task -4:

Prompt: The task is to create a RESTful API for an online voting system using Node.js and Express. The API should allow users to view the list of candidates, cast a vote, and view voting results. The system must ensure that each user can vote only once using in-memory validation. The API should handle JSON-based requests and responses for voting operations.

```

Assignment-14.3.html Assignment-15.5.js Assignment-16.3.py Assignment-17.5.py
# Assignment-15.5.js > ...
295 # Task-4 :
296 // Import Express
297 const express = require('express');
298 const app = express();
299
300 // Middleware
301 app.use(express.json());
302
303 // In-memory Data (Candidates)
304 let candidates = [
305   { id: 1, name: "Alice", votes: 0 },
306   { id: 2, name: "Bob", votes: 0 },
307   { id: 3, name: "Charlie", votes: 0 }
308 ];
309
310 // Store users who have voted
311 let votedUsers = new Set();
312
313 // Home route
314 app.get('/', (req, res) => {
315   res.send("Online Voting API is running...");
316 });
317
318 // GET /candidates -> List candidates
319 app.get('/candidates', (req, res) => {
320   res.json(candidates);
321 });
322
323 // POST /vote -> Cast a vote
324 app.post('/vote', (req, res) => {
325   const { userId, candidateId } = req.body;
326
327   // Validate input
328   if (!userId || !candidateId) {
329     return res.status(400).json({
330       message: "userId and candidateId are required"
331     });
332   }
333
334   // Check if user already voted
335   if (votedUsers.has(userId)) {
336     return res.status(403).json({
337       message: "User has already voted"
338     });
339   }
340
341   // Find candidate
342   const candidate = candidates.find(c => c.id === candidateId);
343
344   if (!candidate) {
345     return res.status(404).json({
346       message: "Candidate not found"
347     });
348   }
349

```

Output:

```

Pretty-print
[{"id":1,"name":"Alice","votes":0}, {"id":2,"name":"Bob","votes":0}, {"id":3,"name":"Charlie","votes":0}]

```

Justification:**Justification:**

This project demonstrates how RESTful APIs can be used to implement an online voting system with basic security validation. It helps in understanding API development, data validation, and preventing duplicate voting using in-memory storage. The system simulates a secure voting process and result calculation. This project is useful for learning backend API development and voting system logic.

Task – 5:

Prompt: The task is to develop a RESTful API to manage teacher records using Node.js and Express. The API should allow users to list all teachers, add new teacher records, update teacher details, and delete teacher records. The system must handle JSON-based requests and responses and include proper error handling for invalid teacher IDs or missing data. The API should support CRUD operations for teacher management.

```
Assignment-14.3.html JS Assignment-15.5.js X Assignment-16.3.py Assignment-17.3.py
JS Assignment-15.5.js > ...
371
372 # Task - 5:
373 // Import Express
374 const express = require('express');
375 const app = express();
376
377 // Middleware to parse JSON
378 app.use(express.json());
379
380 // In-memory data (Candidates)
381 let candidates = [
382   { id: 1, name: "Alice", votes: 0 },
383   { id: 2, name: "Bob", votes: 0 },
384   { id: 3, name: "Charlie", votes: 0 }
385 ];
386
387 // Store users who have voted
388 let votedUsers = new Set();
389
390 // Home route
391 app.get('/', (req, res) => {
392   res.send("Online Voting API is running...");
393 });
394
395 // GET /candidates - List candidates
396 app.get('/candidates', (req, res) => {
397   res.json(candidates);
398 });
399
400 // POST /vote - Cast a vote
401 app.post('/vote', (req, res) => {
402   const { userId, candidateId } = req.body;
403
404   // Validate input
405   if (!userId || !candidateId) {
406     return res.status(400).json({
407       message: "userId and candidateId are required"
408     });
409   }
410
411   // Check if user already voted
412   if (votedUsers.has(userId)) {
413     return res.status(403).json({
414       message: "User has already voted"
415     });
416   }
417
418   // Find candidate
419   const candidate = candidates.find(c => c.id === candidateId);
420
421   if (!candidate) {
422     return res.status(404).json({
423       message: "Candidate not found"
424     });
425   }
426
427   // Increment votes
428   candidate.votes++;
429
430   // Add user to votedUsers
431   votedUsers.add(userId);
432
433   res.json({ message: "Vote cast successfully" });
434 });
435
```

Output:

```
pretty-print
{"success":true,"data":[{"id":1,"name":"Alice","votes":0}, {"id":2,"name":"Bob","votes":0}, {"id":3,"name":"Charlie","votes":0}], "message":"All candidates listed successfully"}
{"success":true,"message":"Vote cast successfully"}
{"success":false,"message":"User has already voted"}
{"success":false,"message":"Candidate not found"}
{"success":false,"message":"userId and candidateId are required"}

```

Justification:

This project demonstrates how RESTful APIs can be used to manage teacher information using CRUD operations. It helps in understanding API routing, JSON data handling, and error handling in backend development. The system allows efficient management of teacher records such as adding, updating, and deleting information. This project is useful for learning backend API development and data management systems.